

AhamX Bodhi

Where I Becomes Infinite

Probability and Token Prediction

Module 1.2: LLM Fundamentals — Prof. Sashikumaar Ganesan



Session Overview

This Session

- How LLMs assign probabilities to tokens
- The next-token prediction mechanism
- Probability distributions over vocabulary
- Softmax and logit transformation
- From probabilities to generated text

Foundation Required

- Basic probability concepts
- What a Language Model is
- Concept of tokens and vocabulary
- Exposure to neural networks (helpful)



Learning Objectives

By the End of This Session, You Will Be Able To

1. **Explain** how a language model assigns a probability to each token in its vocabulary
2. **Describe** the next-token prediction task and why it is central to LLMs
3. **Interpret** a probability distribution produced by the softmax function
4. **Distinguish** between raw logits and normalised probabilities
5. **Relate** token-prediction probabilities to coherent text generation

Quick Recap: Tokens and Vocabulary



Token Definition

A **token** is the atomic unit of text that a language model processes. Tokens may be whole words, subwords, or individual characters, depending on the tokenizer used (e.g., BPE, WordPiece).

Vocabulary Facts

- A model's **vocabulary** V is the fixed set of all tokens it knows
- Typical vocabulary sizes: 32 000 (Llama) to 100 000+ (GPT-4)
- Every input sequence is represented as an ordered list of token IDs
- The model's task: given past tokens, predict the next one

22 lang + 302.
80k to 128k

$\{x_1, \dots, x_{d-1}\}$
↓
 $[x]$
↓
 $\}_{d+1}$

tokens: $2T \approx 40T$



The Core Task: Next-Token Prediction



Formal Definition

Given a context sequence $x_{1:t} = (x_1, x_2, \dots, x_t)$, the model estimates the **conditional probability distribution**:

$$P(x_{t+1} \mid x_1, x_2, \dots, x_t)$$

over every token $x_{t+1} \in \mathcal{V}$.



Probability Distribution over the Vocabulary

What the Model Outputs

For every forward pass, the model produces a **probability vector** $p \in \mathbb{R}^{|\mathcal{V}|}$ where:

$$p_i = P(\underline{x}_{t+1} = \underline{v_i} \mid \underline{x_{1:t}}), \quad p_i \geq 0, \quad \sum_{i=1}^{|\mathcal{V}|} p_i = 1$$

Key Properties

- Each entry p_i represents how likely token v_i is the next token
- The distribution sums to 1 across all $|\mathcal{V}|$ tokens
- High-probability tokens are contextually likely; low-probability ones are unlikely
- The model does *not* just output a single word—it ranks every possible token

I like — 

Raw Scores: Logits



Logit Definition

Before producing probabilities, the model computes **logits** $z \in \mathbb{R}^{|\mathcal{V}|}$ —unnormalised real-valued scores from the final linear (projection) layer:

$$z = W_{lm} \cdot h_t + b$$

~~Transformer block~~

where h_t is the hidden state at position t , W_{lm} is the language-model head weight matrix.

S.2R

Logit Properties

- Logits can be any real number (positive or negative)
- A higher logit means the model “prefers” that token
- Logits are *not* probabilities; they must be normalised first
- The language-model head maps $d_{model} \rightarrow |\mathcal{V}|$ dimensions

$$y_i = g(x_i) = Wx_i + b$$

x_i	y_i
$\vdots$	



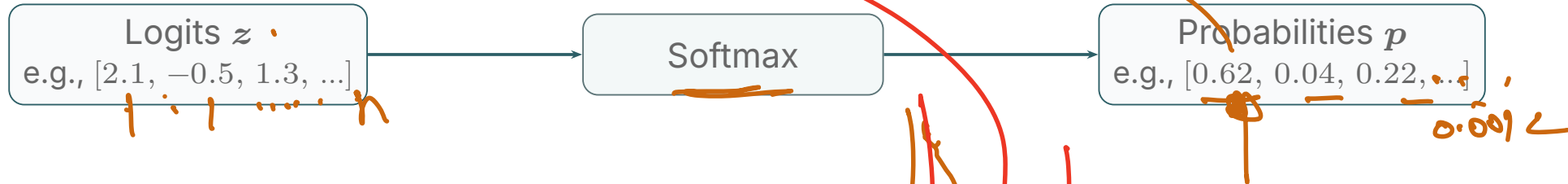
The Softmax Function: Logits → Probabilities

Softmax Definition

The **softmax** function converts logits into a valid probability distribution:

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}}$$

Exponentiating amplifies differences; dividing by the sum normalises to $[0, 1]$.





Worked Example: Softmax in Action

Mini-Example with 4 Tokens

Suppose $\mathcal{V} = \{\text{"cat"}, \text{"dog"}, \text{"sat"}, \text{"the"}\}$ and logits are:

$$\mathbf{z} = [2.0, 1.0, 0.1, -1.0]$$

Compute e^{z_i} : $[7.39, 2.72, 1.10, 0.37]$, Sum = 11.58

Probabilities:

$$\mathbf{p} = [0.638, 0.235, 0.095, 0.032]$$

- "cat" wins with 63.8% probability – highest logit dominates after exponentiation
- "the" is very unlikely (3.2%) – contextually implausible
- All four values are positive and sum to 1.0 – valid distribution

Entropy: How Confident Is the Model?



Shannon Entropy of the Prediction

The model's uncertainty is captured by **entropy**:

$$H(\mathbf{p}) = - \sum_{i=1}^{|\mathcal{V}|} p_i \log p_i$$

Low entropy $\Rightarrow$ confident prediction; High entropy $\Rightarrow$ uncertain.

I like P.

I like the —

I like the cat —

" " " " very —
" " ? " " very much —

Entropy Extremes

- **Peaked distribution** (e.g., $p_1 = 0.99$): entropy ≈ 0 – model is very sure
- **Uniform distribution** (all $p_i = 1/|\mathcal{V}|$): maximum entropy – random guessing
- Entropy links to **perplexity**: $PPL = e^H$ – a common LLM quality metric
- A well-trained model achieves low entropy on in-distribution text

Temperature: Shaping Token Probabilities



Temperature-Scaled Softmax

A **temperature** parameter $T > 0$ is applied to logits before softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Sort $\{x_i\}$ in descending order based on $\{p_i\}$

$T < 1$ (Sharper)

Choose deterministic

- Amplifies differences between logits
- High-probability tokens become *even more* dominant
- Output is more **deterministic** and repetitive
- Use case: factual Q&A, code generation

$T > 1$ (Softer)

- Flattens the distribution toward uniform
- Lower-probability tokens get a chance
- Output is more **diverse** and creative
- Use case: creative writing, brainstorming



Sentence Probability via the Chain Rule

Probability of a Sequence

Using the **chain rule of probability**, the probability of a full sequence $(x_1, x_2, \dots, x_T)$ factorises as a product of conditional probabilities:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$$

Each term is exactly what the LLM computes at step t .

Why This Matters

- The LLM is trained to *maximise* this joint probability on training data
- At inference, it *samples* or *argmax-es* one token at a time
- Cross-entropy loss $= -\log P(\text{target token})$ is the training objective
- The chain rule justifies the autoregressive generation paradigm



Training Objective: Cross-Entropy Loss

Cross-Entropy Loss Definition

The model is trained by minimising the **average negative log-likelihood** over the training corpus of N tokens:

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^N \log P(x_t \mid x_1, \dots, x_{t-1})$$

$P(x_t \mid \dots)$ is the model's predicted probability for the *true* next token x_t .

Intuition

- If the model predicts the true token with $p = 1.0$: loss = 0 (perfect)
- If the model predicts it with $p = 0.01$: loss = $\log(100) \approx 4.6$ (bad)
- Minimising loss $\equiv$ maximising the probability assigned to actual next tokens

From Probabilities to Text: Sampling Strategies



How a Token Is Actually Chosen

Once the probability distribution p is computed, a **decoding strategy** selects the next token. The choice affects quality, diversity, and coherence.

Deterministic

- **Greedy**: always pick $\arg \max p_i$
- **Beam search**: keep top- k partial sequences
- Consistent, but can be repetitive or dull

Stochastic

- **Top- k** : sample from top k tokens
- **Nucleus (Top- p)**: sample from smallest set summing to $\geq p$
- More diverse outputs; controlled by temperature

Autoregressive Generation Pipeline



I →
I like →
I like the →
I like the cat →

Append & repeat

Control token $\leq p$

8K ~ 12K ~ 18K ~ ∞
↑
through
Engo.



Autoregressive Loop

- Each newly generated token is *appended* to the context
- The model runs another forward pass with the extended context
- Generation stops at a special <EOS> token or max length



GenAI Application: Conversational AI (ChatGPT, Claude)

How Probability Drives Chat

Every response word in ChatGPT or Claude is generated via the exact token-prediction mechanism described in this session. The conversation history is the context; the reply is generated token-by-token.

Behind the Scenes

- User message + system prompt = context $x_{1:t}$
- Model runs softmax to produce $P(\text{next token} \mid \text{context})$ at each step
- Temperature settings control creativity vs. precision of responses
- Billions of such predictions are executed per second across all users
- The model's "understanding" is entirely encoded in these probability estimates



GenAI Application: Code Completion (GitHub Copilot)

Code as a Token Sequence

Source code is tokenised exactly like natural language. GitHub Copilot (powered by OpenAI Codex) predicts the next code token given the file context—function signatures, comments, prior code.

Probability in Code Generation

- High probability assigned to syntactically and semantically valid continuations
- Low temperature used to favour correct, deterministic completions
- The model learns that after `def foo(x):` the next tokens are likely indented code
- **Nucleus sampling** used to offer multiple suggestions (diverse p values)
- Same softmax mechanism as text—language is language, code or prose



GenAI Application: Machine Translation (DeepL, Google Translate)

Token Prediction Across Languages

Neural machine translation generates the target-language sequence one token at a time, predicting each target token conditioned on the source text and all previously generated target tokens.

Probability at Work in Translation

- Source: "Bonjour le monde" → Context encoding
- Target token 1: $P(\text{"Hello"} \mid \text{source}) = 0.91$ – high confidence
- Target token 2: $P(\text{"the"} \mid \text{source, "Hello"}) = 0.78$
- **Beam search** explores multiple high-probability translation paths simultaneously
- Final output is the sequence with the highest joint probability $\prod_t P(x_t \mid \dots)$



GenAI Application: Text Summarisation (Claude, GPT-4)

Abstractive Summarisation via Prediction

Abstractive summarisers such as Claude and GPT-4 generate each word of the summary by predicting the most probable next token given the original document and the partial summary produced so far.

Key Observations

- The entire source document forms the context window
- The model assigns high probability to concise, informative continuations
- Unlike extractive methods, the model can *paraphrase*—new tokens not in source
- Low temperature ensures factual fidelity; higher temperature allows creative phrasing
- **RLHF** fine-tuning steers probability mass towards human-preferred summaries



GenAI Application: Retrieval-Augmented Generation (RAG)

Grounding Token Prediction with Facts

In RAG systems (e.g., Perplexity AI, enterprise Q&A bots), retrieved documents are prepended to the context. The LLM then predicts tokens that are grounded in retrieved evidence, reducing hallucination.

How Probability Shifts with RAG

- Retrieved context shifts probability mass toward factually accurate tokens
- Without retrieval: $P(\text{"1969"} \mid \text{"Moon landing year?"}) \approx 0.6$
- With retrieval (NASA doc): $P(\text{"1969"} \mid \text{context+doc}) \approx 0.97$
- RAG does not change the model's weights—it changes the *context* that drives prediction
- Core mechanism: still softmax over $|\mathcal{V}|$ tokens, now with richer context

Common Misconceptions



Clear These Up!

- **Myth:** LLMs “understand” meaning the way humans do
Fact: They assign probabilities based on statistical patterns in training data
- **Myth:** The model always picks the highest-probability token
Fact: Sampling strategies introduce controlled randomness; greedy is just one option
- **Myth:** A higher temperature makes the model smarter
Fact: Higher temperature only broadens the distribution—it doesn’t add knowledge
- **Myth:** Probabilities close to 1.0 mean the model is always correct
Fact: A high-confidence wrong prediction (hallucination) is still a wrong prediction



Skills and Professional Relevance

Technical Skills Gained

- Reading and interpreting logit/softmax outputs
- Configuring temperature and sampling parameters in APIs
- Understanding cross-entropy loss in training logs
- Evaluating model quality using perplexity
- Debugging generation issues (repetition, hallucination)

Roles That Use This Knowledge

- **ML Engineer:** Tunes sampling for production inference
- **AI Engineer:** Configures LLM API temperature settings
- **Data Scientist:** Interprets perplexity and loss curves
- **Research Scientist:** Studies distribution shifts and alignment
- **Product Manager:** Sets quality expectations for generative features



Knowledge Graph: Concept Positioning

This Concept Requires

- What is a Language Model? (AI-LM-FN-TH-000001)
- Basic probability theory (conditional probability)
- Understanding of tokens and vocabulary

This Concept Enables

- Autoregressive Generation (AI-LM-FN-TH-000003)
- Temperature and Top-P parameters (AI-LM-IN-TH-000002/3)
- Perplexity and Evaluation Metrics (AI-SA-IN-TH-000004)
- Cross-entropy loss in fine-tuning (AI-FT-IN-TH-000001)
- RLHF reward modelling (AI-RL-IN-TH-000002)

Session Summary



Key Takeaways

1. A language model outputs a **probability distribution** over the entire vocabulary for each next token
2. **Logits** are raw unnormalised scores; **softmax** converts them to valid probabilities
3. The joint probability of a sequence uses the **chain rule**: $P(\mathbf{x}) = \prod_t P(x_t | \mathbf{x}_{<t})$
4. **Temperature** scales logits to control sharpness vs. diversity of the output distribution
5. **Sampling strategies** (greedy, top- k , nucleus) determine how a token is chosen from p
6. Training minimises **cross-entropy loss** $= -\log P(\text{true token})$ over the training corpus



Next Steps in Your Learning Journey

Immediate Next Concepts

- **Autoregressive Generation**
(AI-LM-FN-TH-000003): How text flows from repeated token prediction
- **Model Sizing and Scaling Laws**
(AI-LM-FN-TH-000004): How model size affects prediction quality
- **Foundation Models Overview**
(AI-LM-FN-TH-000005): Real-world LLM families

Hands-On Practice

- Call an LLM API and inspect raw logit outputs
- Experiment with temperature $T \in \{0.1, 0.7, 1.5\}$ and observe distribution shift
- Compute perplexity on a short sentence by hand
- Explore the transformers library's `generate()` arguments

Thank You

Probability and Token Prediction — AI-LM-FN-TH-000002